

Estructuras de control

Esteban Álvarez

Estructuras de control

- Las estructuras de control de flujo determinan el **orden** de ejecución de las instrucciones de un programa
- **Tipos** de estructuras de programación utilizadas para el control del flujo de un programa:
 - Secuencia: Compuestas por una serie de sentencias o instrucciones (0,1,N...) que se ejecutan en el orden que aparecen. Es lo que hemos hecho hasta ahora
 - Selección o condicionales: Se evalúa una sentencia que devuelve un resultado y se ejecuta una instrucción u otra dependiendo de el valor devuelto. Por ejemplo: si 2 es menor que 5 entonces...
 - Iteración o repetición: Se trata de un conjunto de sentencias que se repiten mientras se cumpla una condición: Mientras i sea menor que 5 hacer...

Estructuras de control

- Además de estas estructuras de control veremos las **sentencias de salto** que aunque no son muy recomendables, es necesario conocerlas
- Muchas veces, las estructuras de control **generarán errores** por lo que es interesante conocer el manejo de excepciones en java
- Vídeo [Estructuras de control](#)

Estructura secuencial

Ya sabemos que:

- Las instrucciones se ejecutan **en el orden** en que se escriben
- Cada instrucción finaliza con **punto y coma ;**
- Las instrucciones se suelen agrupar en **bloques** que van entre llaves

```
{  
    instrucción1;  
    instrucción2;  
    instrucciónN;  
}
```

Estructura de selección o condicionales

- Determinan **si se ejecuta una instrucción** o bloque de instrucciones dependiendo de si se cumple o no una determinada **condición**
- El funcionamiento es sencillo:
 - Siempre vamos a tener una **sentencia de decisión**. Por ejemplo “está lloviendo”
 - Esa sentencia se evalúa y **devuelve dos valores** posibles:
 - Verdadero
 - Falso
 - Si está lloviendo (está lloviendo=verdadero) entonces, abro el paraguas
 - Si no (está lloviendo=falso), guardo el paraguas en la mochila

Estructura de selección o condicionales

- En java la estructura condicional se implementa mediante
 - Estructura de selección simple **if**
 - Estructura de selección compuesta **if-else**
 - Estructura de selección múltiple **switch**

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

```
if(condición){  
    instrucción1;  
    instrucción2;  
}  
instrucción3;
```

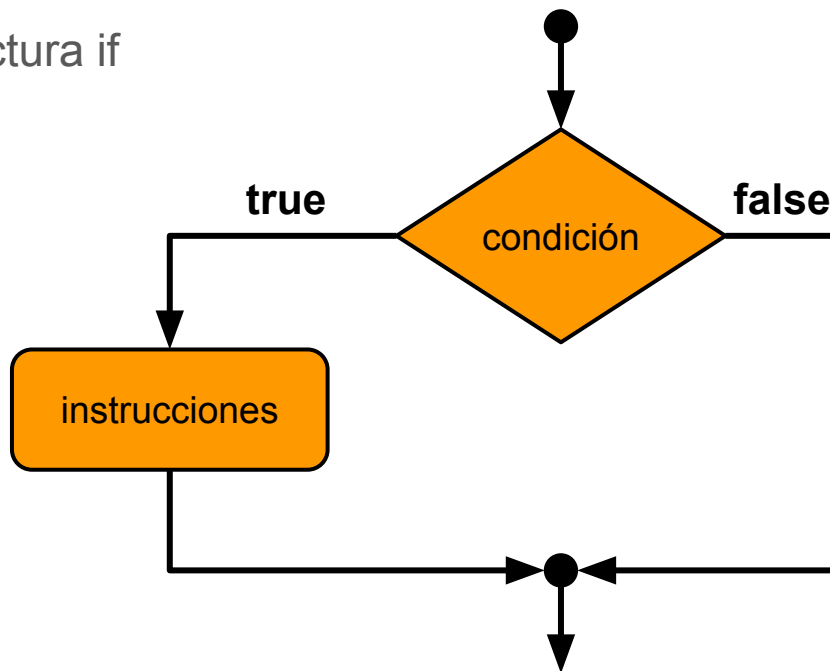
- Se evalúa una **condición**
 - **Si se cumple**, se ejecuta el bloque de instrucciones (instrucción 1 y 2). A continuación se ejecutaría la instrucción3
 - **Si no se cumple**, se salta el bloque de instrucciones. Se ejecutaría solo la instrucción3

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

- Diagrama de flujo de la estructura if

```
if(condición){  
    instrucción1;  
    instrucción2;  
}  
instrucción3;
```



Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

IMPORTANTE: el if se evalúa siempre a true o false

```
if ( false ){  
    instrucción1;  
    instrucción2;  
}  
instrucción3;
```

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

```
System.out.println("Antes del if");  
if(2<3){  
    System.out.println("2 es menor que 3");  
}  
System.out.println("Después del if");
```

- ¿Qué se muestra por pantalla?

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

```
System.out.println("Antes del if");  
if(2<3){  
    System.out.println("2 es menor que 3");  
}  
System.out.println("Después del if");
```

- ¿Qué se muestra por pantalla?
 - Antes del if
 - 2 es menor que 3
 - Después del if

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

```
System.out.println("Antes del if");  
if(3<2){  
    System.out.println("3 es menor que 2");  
}  
System.out.println("Después del if");
```

- ¿Qué se muestra por pantalla?

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

```
System.out.println("Antes del if");  
if(3<2){  
    System.out.println("3 es menor que 2");  
}  
System.out.println("Después del if");
```

- ¿Qué se muestra por pantalla?
 - Antes del if
 - Después del if

Estructura de selección o condicionales

EJERCICIO

- Ejecuta el siguiente código y analiza el resultado:

```
System.out.println("Antes del if");  
if(true){  
    System.out.println("Dentro del if");  
}  
System.out.println("Después del if");
```

- Ejecuta ahora **if**(false) y analiza el resultado

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple. **Sintaxis**

```
if(2<3){  
    System.out.println("2 es menor que 3");  
}
```

- A continuación del **if** la condición va **entre paréntesis**

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple. **Sintaxis**

```
if(2<3){  
    System.out.println("2 es menor que 3");  
}
```

- El bloque de instrucciones que se ejecuta cuando se cumple la condición **va entre llaves**
- Cuando en el bloque de instrucciones solo hay **una sentencia** no es obligatorio usar las llaves
- Aun así, se **recomienda** escribir siempre llaves para evitar confusiones

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

- Buenas prácticas:
 - Escribir el if, la condición y la llave de apertura en la misma línea

```
if(2<3){  
    System.out.println("2 es menor que 3");  
    System.out.println("Fin!");  
}
```

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

- Buenas prácticas:
 - Las instrucciones van en las siguientes líneas, cada una en una línea y tabuladas a la derecha del if

```
if(2<3){  
    System.out.println("2 es menor que 3");  
    System.out.println("Fin!");  
}
```

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

- Buenas prácticas:
 - La llave de cierre se ubica en otra línea y a la misma altura que el if

```
if(3<2){  
    System.out.println("3 es menor que 2");  
    System.out.println("Fin!");  
}
```

Estructura de selección o condicionales

ESTRUCTURA IF. Condicional simple

- Cualquier cosa parecido a lo siguiente tendrá una **calificación de 0 puntos** en un examen:

```
if  
(2<3){  
System.out.println("2 es menor que 3");  
                System.out.println("Fin!");    }
```

- **TRUCO:** Pulsando Ctrl+Mayus+F en Eclipse, obtenemos un autoformato del código

Estructura de selección o condicionales

EJERCICIOS

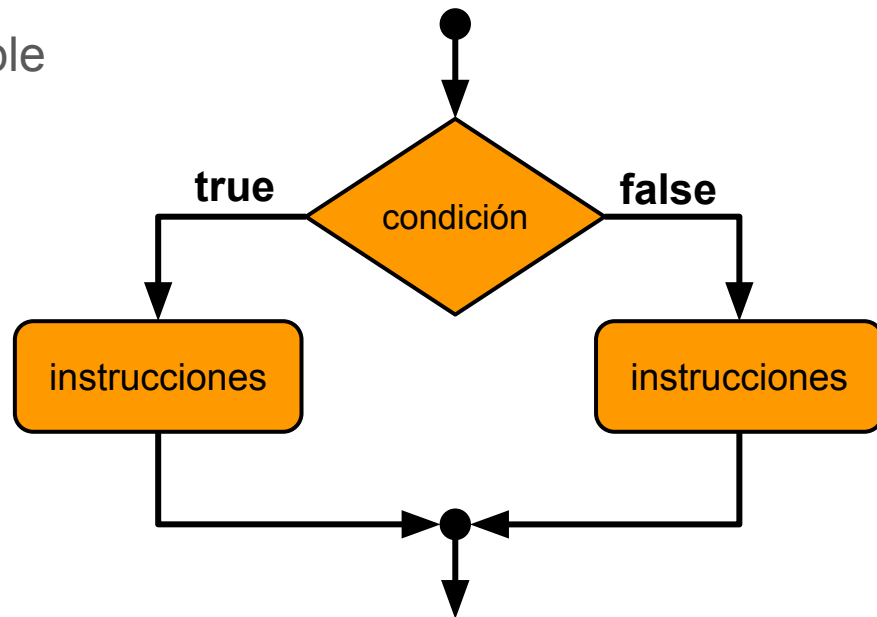
1. Programa que pida una nota por teclado y muestre un mensaje de enhorabuena si el alumno ha aprobado
2. Programa que pida la nota de un examen y la de un trabajo por teclado y muestre un mensaje de enhorabuena si el alumno ha aprobado teniendo en cuenta que la nota del examen cuenta un 80% y la del trabajo el 20% restante
3. Programa que pida por teclado un nombre de usuario y si es igual a “admin” muestre un mensaje indicando que es el administrador del sistema

* Mostrar siempre el mensaje “Fin del programa”

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE. Condicional doble

```
if ( false ) {  
    instrucción1;  
}  
else {  
    instrucción2;  
}
```



- Se **evalúa** la condición
 - **Si se cumple** (true), se ejecuta el primer bloque de instrucciones (instrucc1)
 - **Si no se cumple** (false), se ejecuta el segundo bloque (instrucc2)

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE. Condicional doble

- Buenas prácticas (además de las anteriores):
 - Se escribe la sentencia else y la llave de apertura en la misma línea
 - El else está alineado con su if correspondiente

```
if(condición){  
    instrucción1;  
}else{  
    instrucción2;  
}
```

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE. Condicional doble

- Buenas prácticas (además de las anteriores):
 - El else está alineado con su if correspondiente (o en la misma línea de la llave de cierre del if)
 - La llave de cierre se ubica en otra línea y a la misma altura que el else

```
if(condición){  
    instrucción1;  
}else{  
    instrucción2;  
}
```


Estructura de selección o condicionales

ESTRUCTURA IF-ELSE. Condicional doble

Ejercicios:

1. Modifica los ejercicios anteriores para mostrar un mensaje en caso de que no se cumpla la condición
2. Programa que pida una edad por teclado y muestre si es o no mayor de edad
3. Programa que pida por teclado un número entero e indique si es par o impar (Un número es par cuando el resto de su división entre 2 es 0)

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE. Condicional doble

Uso de if-else vs. uso de if:

- Haz un programa que diga si un número es mayor o igual que 5 o menor que 5:
 - Con un if-else
 - Con dos if
 - Usamos if-else para contemplar todos los casos del problema: el if se encarga de una parte y el else de todos los restantes
- | | |
|----|------|
| if | else |
|----|------|
- Pero cuidado porque también podemos usar dos if: uno que se encarga de la parte if y otro de la parte else
 - **Siempre** se recomienda el uso de if-else en lugar de dos if

Estructura de selección o condicionales

OPERADORES LÓGICOS

- Se pueden “unir” varias comparaciones dentro del if utilizando los operadores AND (&&) y OR (||)

EJERCICIO: Programa que pida al usuario una nota con decimales

- La nota debe estar entre 0 y 10 (incluidos). Mostrar un mensaje indicando si la nota es correcta o no
- Una vez asegurados de que la nota es correcta, mostrar un mensaje indicando si ha aprobado o suspendido

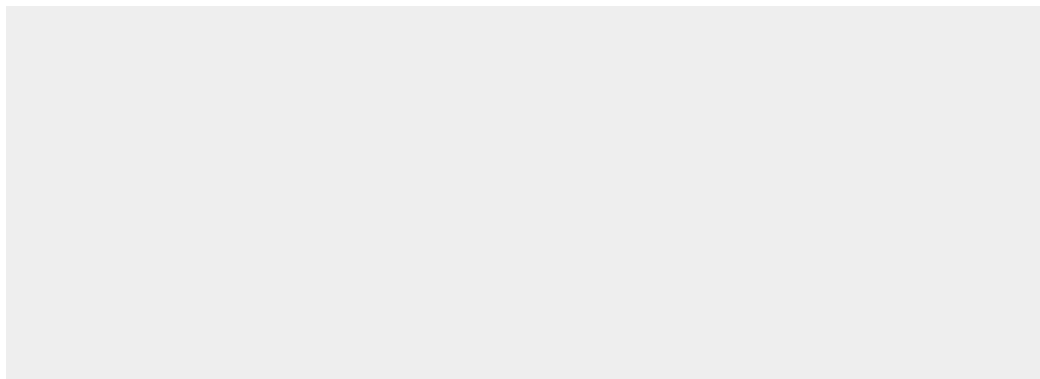
Estructura de selección o condicionales

```
double nota=teclado.nextDouble();
if(nota>=0 && nota<=10) { //Nota válida
    if(nota<5) {
        System.out.println("Suspenso");
    }else {
        System.out.println("Aprobado");
    }
}else { //Nota fuera del rango 0-10
    System.out.println("Nota incorrecta");
}
```

Estructura de selección o condicionales

- Esto es un ejemplo perfecto de **validación de datos** de entrada:

```
double nota=teclado.nextDouble();  
if(nota>=0 && nota<=10) { //Nota válida
```



Si la nota es válida,
hago esta tarea

```
}else {//Nota fuera del rango 0-10  
    System.out.println("Nota incorrecta");  
}
```

Estructura de selección o condicionales

IF CON BOOLEAN

- Suele ser habitual el uso de la estructura **if con variables booleanas**
- Ejemplo: programa que pregunte al usuario si está lloviendo. Si llueve, mostrar el mensaje “Abriendo paraguas” y si no llueve, “Cerrando paraguas”

```
System.out.println("¿Está lloviendo?");  
boolean llueve=teclado.nextBoolean();  
if(????) {
```

Estructura de selección o condicionales

IF CON BOOLEAN

```
if(llueve==true) {  
    System.out.println("Abriendo paraguas");  
}else {  
    System.out.println("Cerrando paraguas");  
}
```

- llueve==true funciona pero no es la forma más correcta de hacerlo

Estructura de selección o condicionales

IF CON BOOLEAN

- `llueve==true` funciona pero no es la forma más correcta de hacerlo:
 - Si llueve vale true: `llueve==true=true`

```
if(    true    ) {  
    System.out.println("Abriendo paraguas");  
}else {  
    System.out.println("Cerrando paraguas");  
}
```


Estructura de selección o condicionales

IF CON BOOLEAN

- Pero si llueve ya vale true, sobra la comparación:

```
if(    true    ) {  
    System.out.println("Abriendo paraguas");  
}else {  
    System.out.println("Cerrando paraguas");  
}
```

Estructura de selección o condicionales

IF CON BOOLEAN

- Ejemplo de lo que pasa cuando llueve vale false:

```
if(    false    ) {  
    System.out.println("Abriendo paraguas");  
}else {  
    System.out.println("Cerrando paraguas");  
}
```

Estructura de selección o condicionales

IF CON BOOLEAN

- Lo anterior se lee de la siguiente manera:

Si llueve, muestro el mensaje “Abriendo paraguas” y si no...

```
if(llueve) {  
    System.out.println("Abriendo paraguas");  
}else {  
    System.out.println("Cerrando paraguas");  
}
```

Estructura de selección o condicionales

IF CON BOOLEAN. INVERSIÓN LÓGICA

- ¿Y cómo codificamos el problema:

*“Si **no llueve** muestro el mensaje ‘Cerrando paraguas’ y si no ...”*

Estructura de selección o condicionales

IF CON BOOLEAN. INVERSIÓN LÓGICA

- ¿Y cómo codificamos el problema:

*“Si **no llueve** muestro el mensaje ‘Cerrando paraguas’ y si no ...”*

```
if(!llueve) {  
    System.out.println("Cerrando paraguas");  
}else {  
    System.out.println("Abriendo paraguas");  
}
```

Estructura de selección o condicionales

IF CON BOOLEAN. NOMBRE DE LAS VARIABLES

- La **lectura correcta** de una sentencia if con boolean depende al 100% del nombre que hayamos elegido para la variable:

 `if( lloviendo ) { //Si lloviendo¿?¿?`

 `if( estaLloviendo ) { //Si está lloviendo`

TRUCO: para dar nombre a una variable booleana, piensa cómo la vas a usar luego en un if

Estructura de selección o condicionales

IF CON BOOLEAN

CONCLUSIÓN: en una estructura if, nunca usar operadores de comparación con una variable booleana

```
if( llueve ) {   
    System.out.println("Abriendo paraguas");  
}else {  
    System.out.println("Cerrando paraguas");  
}
```

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE IF. Condicional múltiple

```
if(condición1){  
    instrucción1;  
}else if(condición2){  
    instrucción2;  
}else{  
    instrucción3;  
}
```

- Dentro del **else** se incluye una nueva condición **if**
- Ese nuevo if tiene la estructura del if ya conocido (puede tener un else o no)
- El segundo else se tabula a la altura del segundo if
- **CUIDADO CON LAS LLAVES Y LAS TABULACIONES!!!**

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE IF. Condicional múltiple

Ejercicios:

1. Programa que lea una hora (número entero) y salude según la hora introducida
IMPORTANTE: Prohibido usar los operadores lógicos AND y OR
 - a. “Buenos días” hasta las 12 horas
 - b. “Buenas tardes” de las 12 a las 21 horas
 - c. “Buenas noches” a partir de las 21 horas

Estructura de selección o condicionales

```
if(hora<12) {  
    System.out.println("Buenos días");  
} else if(hora<21) {  
    System.out.println("Buenas tardes");  
} else { //CUIDADO!! No poner: else if(hora>=21)  
    System.out.println("Buenas noches");  
}
```

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE IF. Condicional múltiple

Ejercicios:

2. Programa que lea una hora (número entero) y salude según la hora introducida
IMPORTANTE: Prohibido usar los operadores lógicos AND y OR
 - a. “Buenos días” desde las 0 a las 12 horas
 - b. “Buenas tardes” de las 12 a las 21 horas
 - c. “Buenas noches” de las 21 a las 24 horas

Se debe mostrar un mensaje de error en caso de introducir una hora incorrecta

Estructura de selección o condicionales

```
if (hora >= 0) { // Validación
    if (hora < 12) {
        System.out.println("Buenos días");
    } else if (hora < 21) {
        System.out.println("Buenas tardes");
    } else if (hora <= 24) {
        System.out.println("Buenas noches");
    } else { // Hora >24
        System.out.println("Hora incorrecta");
    }
} else { // Hora <0
    System.out.println("Hora incorrecta");
}
```

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE IF. Condicional múltiple

Ejercicio:

3. Programa que pida al usuario un número entre el 0 y el 10 y diga si es mayor, menor o igual que 5.
 - Obligatorio validar que el número esté entre 0 y 10
 - Obligatorio utilizar if-else-if para saber si es mayor, menor o igual que 5

Estructura de selección o condicionales

ESTRUCTURA IF-ELSE IF. Condicional múltiple

Ejercicio:

4. Programa que genere un número aleatorio entre el 0 y el 10 y diga si es suspenso, aprobado o sobresaliente. Obligatorio utilizar if-else-if con una condición simple (sin operadores lógicos)

Estructura de selección o condicionales

ERRORES HABITUALES

- Utilizar el operador = como operador de comparación:

```
if(hora=21) {
```

Type mismatch: cannot convert from int to boolean
Press 'F2' for focus

- Omitir la variable al usar operadores AND y OR:

```
if(hora>=0 && <=12) {
```

The operator && is undefined for the argument type(s) boolean, int
Press 'F2' for focus

- Añadir ; al final del if. Cuidado porque no genera error!

```
if (hora>=0);{
```

```
}
```

Estructura de selección o condicionales

EJERCICIOS

Hacer las actividades de la estructura IF del Campus Virtual

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Se utiliza para seleccionar una de **entre múltiples** alternativas

```
switch (expresión) {  
    case valor 1:  
        Instrucciones;  
        break;  
    case valor 2:  
        Instrucciones;  
        break;  
    . . .  
    default:  
        Instrucciones;  
}
```

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Se utiliza para seleccionar una de **entre múltiples** alternativas

```
switch (expresión) {  
    case valor 1:  
        Instrucciones;  
        break;  
    case valor 2:  
        Instrucciones;  
        break;  
    . . .  
    default:  
        Instrucciones;  
}
```

Primero se evalúa “expresión” que suele ser una variable que contiene un valor de tipo **char**, **int** o **String**

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Se utiliza para seleccionar una de **entre múltiples** alternativas

```
switch (expresión) {  
  case valor 1:  
    Instrucciones;  
    break;  
  case valor 2:  
    Instrucciones;  
    break;  
  . . .  
  default:  
    Instrucciones;  
}
```

A continuación **se salta al case cuyo valor coincida** con el de la expresión

Se ejecutan las instrucciones hasta llegar al break

IMPORTANTE!! Es “obligatorio” poner break al final de cada case o se ejecutarán todos los case siguientes

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Se utiliza para seleccionar una de **entre múltiples** alternativas

```
switch (expresión) {  
  case valor 1:  
    Instrucciones;  
    break;  
  case valor 2:  
    Instrucciones;  
    break;  
  . . .  
  default:  
    Instrucciones;  
}
```

Si **ninguno** de los casos se cumple, se ejecutan las instrucciones del bloque default

El bloque default es **opcional**

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Ejemplo

```
char character='S':  
switch (character) {  
case 'N':  
    Instrucciones;  
    break;  
case 'S':  
    Instrucciones;  
    break;  
default:  
    Instrucciones;  
}
```

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Una forma de ver el switch es la siguiente:

```
char character='S':  
switch (character) {  
if(carácter=='N')  
    Instrucciones;  
    break;  
if(carácter=='S')  
    Instrucciones;  
    break;  
default:  
    Instrucciones;  
}
```

Estructura de selección o condicionales

ESTRUCTURA SWITCH

- Otro ejemplo

```
char caracter='J':  
switch (caracter) {  
    case 'N':  
        Instrucciones;  
        break;  
    case 'S':  
        Instrucciones;  
        break;  
    default:  
        Instrucciones;  
}
```

Estructura de selección o condicionales

ESTRUCTURA SWITCH

Ejercicios:

1. Programa que lee por teclado un mes (número entero) y muestra su nombre
2. Modifica el ejercicio anterior para que muestre el nombre del mes y el de todos los meses siguientes hasta diciembre (incluido). Pista: trabajar sobre el break de los case
3. Programa que lee por teclado un día de la semana (nombre en minúsculas y sin tildes) y muestra su número (lunes->1, martes->2, etc.). En caso de no ser un día de la semana válido, muestra un error

Estructura de selección o condicionales

ESTRUCTURA SWITCH

Es posible agrupar varios case en una misma línea:

```
switch (nota) {  
  case 0,1,2,3,4:  
    System.out.println("Suspenso");  
    break;  
  
  case 5,6,7,8,9,10:  
    System.out.println("Aprobado");  
    break;  
  
  default:  
    System.out.println("Nota no válida");  
}
```

Estructura de selección o condicionales

ESTRUCTURA SWITCH

Ejercicios:

4. Programa que pida al usuario un número entero entre el 0 y el 10 y muestre la calificación en formato texto de la siguiente manera:

- a. Si no está entre el rango de 0 a 10: nota no válida.
- b. Si es 10: matrícula de honor.
- c. Si está entre 9 y 10: sobresaliente.
- d. Si está entre 7 y 9: notable.
- e. Si está entre 6 y 7: bien.
- f. Si está entre 5 y 6: suficiente.
- g. El resto de notas: insuficiente.

Estructura de selección o condicionales

ESTRUCTURA SWITCH

Ejercicios:

Hacer las actividades de la estructura SWITCH del Campus Virtual

Estructura iterativas o repetitivas

- Permiten ejecutar **repetidamente** un **bloque de instrucciones**
- El bucle se ejecuta hasta que se cumpla una condición: **condición de salida**

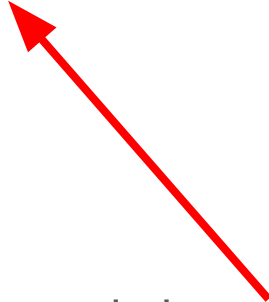
- Tipos de estructuras
 - **for**
 - **while**
 - **do-while**

Estructura iterativas o repetitivas

BUCLE FOR

- Permite la ejecución de un bloque un **número determinado** de veces

```
for (inicialización; condición; incremento/decremento){  
    Instrucción 1;  
    .....  
    Instrucción N;  
}
```



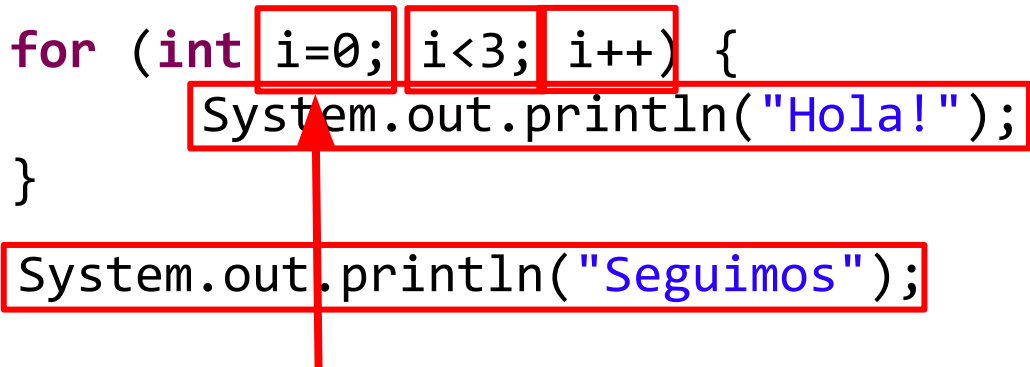
- El bucle se ejecuta mientras se cumpla la condición
- La variable de control se incrementa/decrementa automáticamente
- Los tres parámetros del for están separados por ;

Estructura iterativas o repetitivas

BUCLE FOR

- Permite la ejecución de un bloque un **número determinado** de veces

```
for (int i=0; i<3; i++) {  
    System.out.println("Hola!");  
}  
System.out.println("Seguimos");
```



i es la **variable de control**

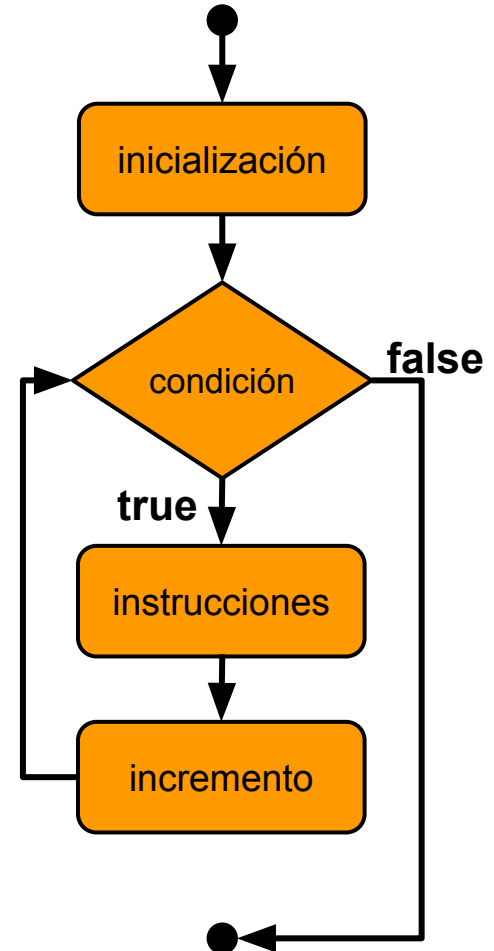
Esta instrucción sólo se ejecuta la primera vez

i	i<3	Pantalla
0	true	Hola!
1	true	Hola!
2	true	Hola!
3	false	Seguimos

Estructura iterativas o repetitivas

BUCLE FOR

```
for (int i=0; i<3; i++) {  
    System.out.println("Hola!");  
}
```



Estructura iterativas o repetitivas

BUCLE FOR

```
for (int i=0; i<3; i++) {  
    System.out.println(i);  
}
```

- ¿Qué se muestra por pantalla?

Estructura iterativas o repetitivas

BUCLE FOR

```
for (int i=0; i<3; i++) {  
    System.out.println(i);  
}
```

- ¿Qué se muestra por pantalla?

0

1

2

Estructura iterativas o repetitivas

BUCLE FOR

IMPORTANTE: la variable `i` es local al bucle for y **se destruye** (desaparece) una vez que finaliza su ejecución

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}  
System.out.println(i);
```

 i cannot be resolved to a variable

4 quick fixes available:

-  [Create local variable 'i'](#)
-  [Create field 'i'](#)
-  [Create parameter 'i'](#)

Estructura iterativas o repetitivas

BUCLE FOR

El bucle for es útil cuando tenemos que **repetir N veces la misma tarea**

EJERCICIOS

1. Pide un número al usuario y muestra el mensaje “Has introducido el número N”
2. Pide 5 números al usuario y muestra el mensaje anterior
 - ~~Puedes copiar y pegar el código anterior 5 veces~~
 - O hacer un for que **repita 5 veces** la tarea del ejercicio 1
3. Pide 500 números al usuario y muestra el mensaje
 - ~~Puedes copiar y pegar el código anterior 500 veces~~ (1500 líneas de código!!)
 - O hacer un for que **repita 500 veces** la tarea del ejercicio 1 (4 líneas de código)

Estructura iterativas o repetitivas

BUCLE FOR

EJERCICIOS

4. Muestra todos los números del 1 al 100
5. Muestra todos los números del 100 al 0
6. Muestra todos los números del 0 al 100 de 8 en 8
7. Muestra 20 números aleatorios enteros del 0 al 10
8. Pide al usuario un número y muéstralo en pantalla una cantidad aleatoria de veces, comprendida entre 5 y 10

Estructura iterativas o repetitivas

BUCLE FOR. COMBINACIÓN CON IF

- Es muy habitual utilizar el bucle **for junto con el if**
- EJERCICIO: Programa que pida 3 números al usuario y, para cada número, muestre si es positivo o negativo
 - Descomponer el problema en problemas más sencillos
 - Pedir un número al usuario y mostrar si es positivo o negativo
 - Repetir la tarea anterior 3 veces

Estructura iterativas o repetitivas

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Introduce un número");  
    double num=teclado.nextDouble();  
    if (num>0) {  
        System.out.println(num+" es positivo");  
    }else {  
        System.out.println(num+" es negativo");  
    }  
}  
System.out.println("Fin del programa");
```

Estructura iterativas o repetitivas

BUCLE FOR

EJERCICIOS

1. Modifica el ejercicio anterior para que pida los números al usuario de la siguiente manera. Ejemplo de ejecución:

```
Escribe tres números.  
Introduce el número 1: 8  
Positivo  
Introduce el número 2: 3  
Positivo  
Introduce el número 3: -5  
Negativo
```

Estructura iterativas o repetitivas

BUCLE FOR

EJERCICIOS

2. Programa que pida 10 números al usuario y muestre si son positivos, negativos o cero (if-else-if)
3. Programa que pida 5 números al usuario y diga si son pares ($\text{número} \% 2 = 0$) o impares

Estructura iterativas o repetitivas

BUCLE FOR

IMPORTANTE: el bucle for se **utiliza** cuando se conoce el número de repeticiones que se quieren hacer

- EJERCICIO: Programa que pregunte al usuario cuántas palabras quiere introducir. Luego pedir que introduzca ese número de palabras y mostrar por pantalla si cada una es igual (o no) a la palabra “Hola”

- El **programador no conoce** a priori el número de iteraciones

```
for (int i = 0; i < ???; i++) {
```

- Pero hará un for con el **número de iteraciones que decida el usuario**

```
int repeticiones=teclado.nextInt();  
for (int i = 0; i < repeticiones; i++) {
```

Estructura iterativas o repetitivas

BUCLES FOR ANIDADOS

- Otras veces se utilizarán **bucles for anidados** (unos dentro de otros)
- EJERCICIO: Programa que pida 5 números al usuario y, para cada número, muestre todos los números que hay entre él y el 0
 - Ampliación: Muestra los resultados de la siguiente manera:

Los números entre 5 y 0 son:

5, 4, 3, 2, 1

Recuerda, has introducido el número 5

Estructura iterativas o repetitivas

BUCLES FOR ANIDADOS

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Introduce un número");  
    int num=teclado.nextInt();  
    for(int j=num;j>=0;j--) { //No puedo usar i!!  
        System.out.print(j+ " ");  
    }  
    System.out.println(); //Para formatear la salida  
}
```

Estructura iterativas o repetitivas

BUCLES FOR ANIDADOS

- Habría que validar que los números que introduce el usuario sean positivos
- Si no lo son, podemos tener un problema serio (haz la prueba)

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Introduce un número");  
    int num = teclado.nextInt();  
    if (num > 0) {  
        for (int j = num; j >= 0; j--) { // No puedo usar i!!  
            System.out.print(j + " ");  
        }  
        System.out.println(); // Para formatear la salida  
    }  
}
```

Estructura iterativas o repetitivas

BUCLES FOR ANIDADOS

1. Escribe todos los pisos de un edificio de N plantas (N es un número que introduce el usuario) y cada planta tiene los pisos numerados de la A a la D (se puede hacer un for de char).

Ejemplo de salida:

Introduce el número de plantas: 2

1ºA, 1ºB, 1ºC, 1ºD

2ºA, 2ºB, 2ºC, 2ºD

Estructura iterativas o repetitivas

BUCLES FOR ANIDADOS

2. Modifica el ejercicio anterior para que su salida sea como la siguiente:

Introduce el número de plantas: 2

Planta 1

1ºA, 1ºB, 1ºC, 1ºD

Planta 2

2ºA, 2ºB, 2ºC, 2ºD

Estructura iterativas o repetitivas

BUCLES FOR ANIDADOS

3. Realiza un programa que muestre las tablas de multiplicar del 1 al 10.

Tabla del 1:

1x1=1

1x2=2

...

Tabla del 2:

2x1=2

2x2=4

...

Estructura iterativas o repetitivas

BUCLE WHILE

- Se utiliza cuando **no sabemos** el número de veces que se tiene que realizar el bucle.

```
while(condición){  
    sentencia1;  
    ...  
    sentenciaN;  
}
```

- Las instrucciones se repiten mientras la condición sea cierta
- La condición **se evalúa antes de entrar** en el bucle por lo que las acciones se pueden ejecutar 0 o más veces
- La variable de control **no se modifica** automáticamente como en el for

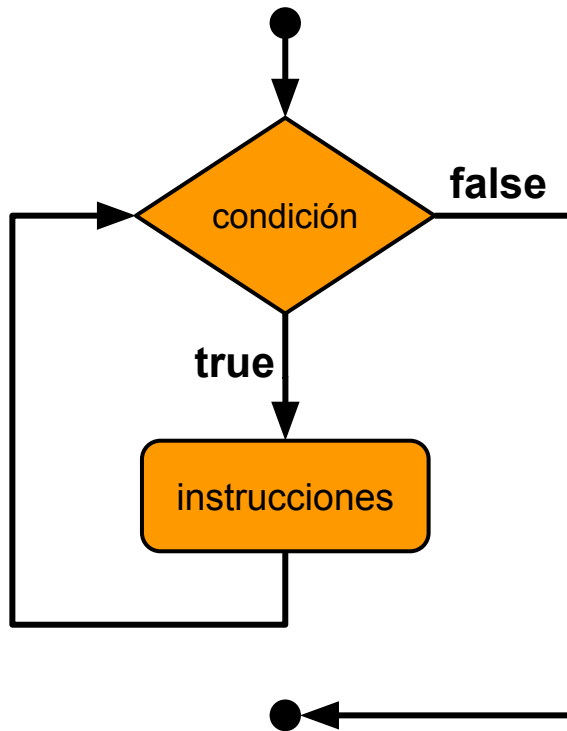
Estructura iterativas o repetitivas

BUCLE WHILE

```
while(condición){  
    sentencia1;  
    ...  
    sentenciaN;  
}
```

Funcionamiento:

1. Se evalúa la condición
 - a. Si el resultado es `false` las instrucciones no se ejecutan, el bucle finaliza y sigue la ejecución del programa
 - b. Si el resultado es `true` se ejecutan las instrucciones y se vuelve al paso 1



Estructura iterativas o repetitivas

BUCLE WHILE

```
int i=0;
while(i<3) {
    System.out.println(i);
    i++;
}
```

- Este bucle while es **exactamente igual que el for** del ejemplo
- Se ejecuta el bloque del while **siempre que se cumpla** que $i < 3$

Estructura iterativas o repetitivas

BUCLE WHILE

```
int i=0;
while(i<3) {
    System.out.println(i);
    i++;
}
```

- Pero hemos tenido que **inicializar la variable de control** nosotros

Estructura iterativas o repetitivas

BUCLE WHILE

```
int i=0;
while(i<3) {
    System.out.println(i);
    i++;
}
```

- Y **modificar su valor** en cada iteración

Estructura iterativas o repetitivas

BUCLE WHILE

IMPORTANTE: si no actualizamos la variable de control, se genera un **bucle infinito** porque nunca se cumple la condición de salida ($i < 3$ en este caso)

```
int i=0;
while(i<3) {//Bucle infinito. i siempre es menor que 3
    System.out.println(i);
}
```

Estructura iterativas o repetitivas

BUCLE WHILE

EJERCICIOS

1. Muestra todos los números del 1 al 100
2. Programa que lee un número N y muestra N asteriscos

ACLARACIÓN:

La solución más óptima para estos ejercicios sería utilizando un bucle FOR puesto que conocemos el número de iteraciones a realizar.

Se pide utilizar WHILE solo a efectos didácticos.

Estructura iterativas o repetitivas

BUCLE WHILE EJERCICIO

Programa que pida al usuario números enteros. El programa finaliza cuando introduce el número 0.

Algoritmo:

Pedir un número al usuario

Mientras ese número sea distinto de 0

Mostrar el número introducido

Volver a pedir otro número al usuario

- Este es un ejemplo real de la **necesidad** de uso del bucle while. Sería imposible hacerlo con for

Estructura iterativas o repetitivas

- Solución:

```
System.out.println("Introduce un número");  
int num=teclado.nextInt();  
while(num!=0) {  
    System.out.println("Introduce otro número");  
    num=teclado.nextInt();  
}
```


Estructura iterativas o repetitivas

- Este otro algoritmo **fuerza la entrada** en el while con la inicialización de la variable `num`

```
int num=1; //Inicializo num con un valor != 0
while(num!=0) { //1ª iteración: num=1 -> entra en el bucle
    System.out.println("Introduce un número");
    num=teclado.nextInt();
}
```

Estructura iterativas o repetitivas

BUCLE WHILE

EJERCICIOS

1. Programa que pida al usuario un número e indicar si es positivo o negativo. El proceso se repetirá hasta que se introduzca un 0.

Algoritmo:

```
Leer num
Mientras (num NO sea lo que quiero. (0 en este caso))
    Hacer
        Leer num
Fin Mientras
```

Estructura iterativas o repetitivas

BUCLE WHILE

EJERCICIOS

2. Programa que pida al usuario un nombre de usuario. Este **debe** ser igual a “profesor”.
Una vez que introduzca “profesor” mostrar el mensaje “Acceso concedido”
3. Programa que pida al usuario un nombre de usuario. Este **debe** ser igual a “profesor”, “alumno” o “invitado”

Estructura iterativas o repetitivas

BUCLE WHILE

EJERCICIOS

4. Realizar un juego para adivinar un número. Para ello generar un número aleatorio entre 1 y 20, y luego ir pidiendo números al usuario indicando “mayor” o “menor” según sea mayor o menor con respecto a N. El proceso termina cuando el usuario acierta.
5. Programa que pida al usuario un nombre de usuario y una contraseña. El nombre de usuario debe ser igual a “admin” y la contraseña “123456”

Estructura iterativas o repetitivas

BUCLE WHILE

CONCLUSIÓN: usamos **while** cuando no sabemos el número de iteraciones

Estructura iterativas o repetitivas

BUCLE DO-WHILE

- Es un bucle WHILE pero que queremos que las instrucciones del bucle **se ejecuten al menos una vez**

```
do{  
    sentencia1;  
    ...  
    sentenciaN;  
}while(condición);
```

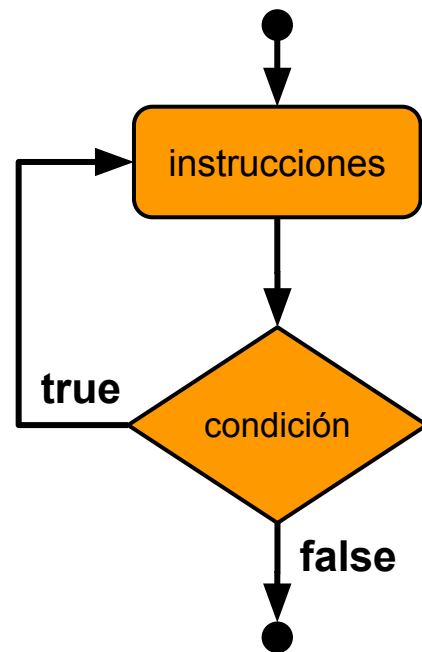
Estructura iterativas o repetitivas

BUCLE DO-WHILE

```
do{  
    sentencia1;  
    ...  
    sentenciaN;  
}while(condición);
```

Ejecución:

1. Se ejecutan las instrucciones a partir de `do{`
2. Se evalúa la `condición`
 - a. Si el resultado es `false`, el bucle finaliza y el programa sigue ejecutándose
 - b. Si es `true`, se vuelve al paso 1



Estructura iterativas o repetitivas

BUCLE DO-WHILE

- **Recuerda este ejercicio:** Programa que pida al usuario números enteros. El programa finaliza cuando introduce el número 0.
- Le dimos solución con while **forzando** la primera ejecución del bucle:

```
int num=1; //Inicializo num con un valor != 0
while(num!=0) { //1ª iteración: num=1 -> entra en el bucle
    System.out.println("Introduce un número");
    num=teclado.nextInt();
}
```


Estructura iterativas o repetitivas

BUCLE DO-WHILE

- Si **necesitamos** que el bucle while se ejecute **al menos UNA vez**, queremos un bucle do-while

```
int num; //Obligatorio declarar num aquí
do{
    System.out.println("Introduce un número");
    num=teclado.nextInt();
}while(num!=0); //Porque si no aquí no existe
```

Estructura iterativas o repetitivas

BUCLE DO-WHILE

Haz los siguientes ejercicios con while y do-while

1. Programa que lea números enteros. El número debe ser menor que 100
2. Programa que lea números entre 0 y 10. Si se introduce un número que no está en ese rango, se finaliza la ejecución del programa

Estructura iterativas o repetitivas

BUCLE WHILE

- Uso típico de un bucle while: **Petición y validación** de datos

EJERCICIO: Programa en java que pida el número de mes al usuario y muestre su nombre. El número del mes debe estar entre 1 y 12

Estructura iterativas o repetitivas

```
System.out.println("Introduce el número del mes");
int numMes=teclado.nextInt();
while(numMes<1 || numMes>12) {
    System.out.println("El número debe estar entre 1 y 12");
    System.out.println("Introduce el número del mes");
    numMes=teclado.nextInt();
}
```

Estructura iterativas o repetitivas

BUCLE WHILE

- Uso típico de un bucle while + switch: Mostrar un **menú de opciones**

EJERCICIO: Programa en java que muestre un menú con las opciones:

1. Introducir operando 1
2. Introducir operando 2
3. Suma
4. Resta
5. Salir

```

boolean salir = false;
int opcion;

while(!salir){
    System.out.println("1. Introducir operando 1");
    System.out.println("2. Introducir operando 2");
    System.out.println("3. Suma");
    System.out.println("4. Resta");
    System.out.println("5. Salir");
    System.out.println("Escribe una de las opciones");

    opcion=teclado.nextInt();

    switch(opcion){
        case 1:
            //Leer operando 1
            break;
        case 2:
            //Leer operando 2
            break;
        case 3:
            //Hacer la suma
            break;
        case 4:
            //Hacer la multiplicación
            break;
        case 5:
            salir=true;
            break;
        default:
            System.out.println("Opción incorrecta");
    }
}

```

Estructuras de salto

BREAK Y CONTINUE

- Java incorpora las sentencias de salto (o de ruptura) **break** y **continue**
- **Modifican** el comportamiento lógico de las estructuras de control vistas

Es necesario conocer estas sentencias pero se **desaconseja su uso**

Estructuras de salto

BREAK

- Se puede incluir en las estructuras de control **iterativas** (for, while, do-while) y en la sentencia **switch**
- Cuando aparece en una sentencia iterativa, se **finaliza totalmente** la ejecución del bucle, pasando a la siguiente instrucción
- El caso del **switch** ya se ha estudiado: evita que se ejecuten los siguientes case

Estructuras de salto

¿Qué sale por pantalla?

```
int dia = 5;
for (int i = 1; i <= 7; i++) {
    if (i == dia) {
        System.out.println("Hoy es el " + i + "º dia de la semana.");
        break;
    }
    System.out.println("Dia " + i);
}
System.out.println("Seguimos...");
```

Estructuras de salto

CONTINUE

- La sentencia continue incide sobre las estructuras **iterativas** for, while y do-while
- Cuando se ejecuta, se **da por finalizada** la iteración actual, pasándose a la siguiente iteración del bucle

Estructuras de salto

¿Qué sale por pantalla?

```
int dia = 5;
for (int i = 1; i <= 7; i++) {
    if (dia == i) {
        continue;
    }
    System.out.println("Dia " + i);
}
System.out.println("Seguimos...");
```

FIN!

(Pasar a la presentación de Contadores, acumuladores y flags)